



Algorithms and Complexity 2026

Exercise Session 5: Flows

This is a collection of problems that hopefully improve your understanding of the material discussed in the lectures. During the exercise classes you have time to work on the first 3-5 problems and will have the opportunity to obtain feedback and/or help from one of the teaching assistants. Towards the end of each session the solutions of these problems will be covered.

To optimally prepare for the mastery test, you are encouraged to solve some of the remaining problems on your own. If you are stuck, require assistance or someone really messed up to write an understandable solution, feel free to send an email to algokomp-help@kth.se or visit an office hour.

Some problems are marked with superscripts: 1^M , 2^* or even 3^{**} . Problems marked with an M superscript are in the spirit of mastery test problems. Such problems are usually not stated as plain math problems but rather consist of a story from which you need to extract the mathematical structure yourself. Problems marked with a $*$ are challenging problems. The more stars the harder – do not get discouraged if you cannot solve such a problem.

1 Altered flow

- (a) Describe an efficient algorithm that finds a new maximum flow if the capacity of a particular edge *increases* by one unit.

Algorithm time complexity should be linear, ie $O(|V| + |E|)$.

- (b) Describe an efficient algorithm that finds a new maximum flow if the capacity of a particular edge *decreases* by one unit.

Algorithm time complexity should be linear, ie $O(|V| + |E|)$.

Solution:

- (a) Suppose the edge from u to v increases its capacity by one. From previous maximum flow Φ , just make a new iteration of Ford-Fulkerson algorithm with the modified graph: The residual flow graph increases the capacity of edge (u, v) by one. Make a graph search (in time $O(|V| + |E|)$) to see if there is any path in the residual flow graph along which the flow can increase. If there is one, it must be a flow of size one (because all flows are integers). If there is no flow in the residual flow graph, Φ is still the maximum flow.
- (b) Suppose the edge from u to v decreases its capacity by one. If previous maximum flow Φ didn't use the full capacity of (u, v) the decrease doesn't count at all. Otherwise the flow is updated as follows:

Since the flow entering u is one unit more than the flow leaving u and the flow entering v is one unit less than the flow leaving v , there is no way to transfer a unit flow from u to v .

Thus, the search in the residual flow graph for a path from u to v along which the flow can increase by one. This is done by a graph visit in time $O(|V| + |E|)$. If such path exists, we update Φ with the flow.

If there is no such path, we must reduce the flow from s to u and from v to t by one unit. We do this by finding a path from u to s in the residual flow graph along which the flow can increase by one and a path from t to v along which flow can increase by one. (There must be such paths, because we had a flow from s to t via (u, v) .) Then, update Φ with these two flows.

2 Sweets

In our garden we have a factory in which we would like to produce sweets. In order to produce a sweet we need to have the required ingredients plus it may require some special tools. The ingredients are either sweets themselves (which we then first have to produce) or some base product of which we have an infinite supply in our garden.

We are given a list of recipes of n different sweets. If a tool is used in several recipes, we only have to buy it once because we may reuse it.

Each sweet s has a pleasure value v_s , which may be negative, and each tool t has an associated cost c_t . If we decide to produce sweets $s_1, s_2, \dots, s_m$ but have to buy tools $t_1, \dots, t_k$ to do so, this gives us a total of $\sum_{i=1}^m v_{s_i} - \sum_{j=1}^k c_{t_j}$ pleasure. The goal is to maximize our pleasure. You may ignore scaling issues: if some sweet s can be produced, you may assume that we have an infinite supply of s .

Design a polynomial time algorithm that finds the best set of sweets to be produced in our factory.

Solution:

This problem is an instance of the closure problem discussed in class.

3^M Sport League

Consider a sport league with n teams that play matches against each other (each match consists of 2 teams). Either a team wins a match or they draw. A win is worth 2 points, a draw 1 point, and losing is not punished nor rewarded.

Suppose that in a single season each team faces each other team exactly once: there are $n(n-1)/2$ matches played in a single season and each team plays in precisely $n-1$ matches.

At the end of each season the teams report their supposed sum of points to us. In order to determine whether a team is blatantly lying, we want to figure out whether there is an outcome of the matches that could have resulted in the reported point distribution.

Design a polynomial time algorithm that, given the supposed point distribution P determines whether there is an outcome of the matches of the season that results in P .

Example: Suppose $n = 3$ and the teams claim that they scored 1, 2 and 3 points. This is indeed a possible outcome of a season: for example if team 1 drew against team 3, team 2 won against team 1 but lost against team 3. On the other hand a reported score of 5, 1, 0 is impossible to achieve.

Solution:

First do a sanity check that P is a point distribution, i.e., that the sum of all reported points is equal to $n(n-1)$. If so, create a bipartite graph $G = (M \cup T, E)$, where M contains a vertex for

each match and T contains a vertex for each team. Add an edge of capacity 2 between a match m and a team t if t plays in m .

Last, connect each match $m \in M$ to a source vertex s by an edge of capacity 2 and each team $t \in T$ to the sink u by an edge of capacity that is equal to the supposed sum of points.

If there is a max flow from s to u of value $n(n-1)$, then there is an outcome of the matches of the season that result in P .

4^M Lona Misa

The criminal organization M wants to smuggle the renowned painting “Lona Misa” from country I to country L . Unfortunately the police in I and L is infiltrated by M and hence is of no help to recover Lona Misa. But it turns out that in order to get from I to L the painting must be smuggled through the country S , which is known for its unbribeable officials. We obtained the information that M will attempt to smuggle Lona Misa by car from I to L .

You are tasked with the job of setting up the checkpoints along the streets of S in order to recover Lona Misa. As the unbribeable officials are quite well-paid, we want to set up as few checkpoints as possible.

Come up with a polynomial time algorithm that, given a map of the streets of S , determines the location and minimum number of required checkpoints in order to recover Lona Misa. You may assume that I and L share borders with S and that M must smuggle Lona Misa from I to S and then from S to L .

Solution:

We can view the map of S as a graph, each edge of capacity 1. Connect all the edges that cross from I to S to a source by an edge of capacity 1, and similarly we connect edges that cross from S to L to the sink.

Running a max flow algorithm on this graph results in the desired mincut. If Ford-Fulkerson has been used, then a min cut can be found as follows. Do a dfs in the residual graph from the source (by only following non-saturated edges). Let V denote all vertices and W the set of visited nodes in the dfs, then a min-cut is given by of all edges going from W to $V - W$.

5[★] Short Winter Season

We are in the hotel business, only host families of 4 and each family may only stay for a single weekend. The n families that would like to stay at our hotel let us know which of the coming k weekends suit them. We want to host all the n families but at the same time would like to close the hotel as soon as possible. We prefer it quiet.

Suppose the hotel can accompany at most m families at once. Your task is to come up with a polynomial time algorithm that determines the least $i \leq k$ such that we can host all the n families on the coming i weekends.

Hint 1: Try to come up with an algorithm that decides whether the n families can be hosted on the k weekends.

Hint 2: Using the algorithm from Hint 1, find the minimum $i \geq k$ for which this holds.

Solution:

A max flow can determine whether we can host all the n families: create a bipartite graph on one side the n families connected to the source by an edge of capacity 1, and on the other hand we have a vertex per weekend connected to the sink by an edge of capacity m . We connect a family to a weekend by an edge of capacity 1 if the weekend suits the family.

Do binary search over the number of weekends. Report the least i such that all families can be accomodated.